Projekt: Data-Mart-Erstellung in SQL

Erstellung einer Datenbank für eine Buchtausch-App

Phase 3 Finalisierungsphase

IU Internationale Hochschule Informatik B.Sc.

11.11.2026, Felix Stellbrink, IU14127320, Tutorin: Anna Androvitsanea

Inhaltsverzeichnis:

1. Überblick 1
2. Funktionalität 1
3. Metadaten 3

1. Überblick

Diese Datenbank wurde für eine App entwickelt, die Nutzer dabei unterstützt, eigene Bücher an andere Personen zu verleihen. Sie enthält Daten über Bücher, Nutzer und Orte. Ein Nutzer stellt zuerst eine Anfrage für ein Buch. Nach einer positiven Rückmeldung wird der Ausleihvorgang gestartet. Es wird vereinbart, ob das Exemplar versendet oder abgeholt wird. Zudem wird eine Ausleihdauer festgelegt. Am Ende kann eine Bewertung vom Ausleiher an den Verleiher abgegeben werden.

Verwendet wurde PostgreSQL in der Version 18.3. Alle Tabellen sowie die Trigger wurden in pgAdmin 4 Version 9.16 getestet.

Eine zentrale Designentscheidung war die Trennung zwischen Buch und Exemplar. Diese Trennung ist nötig, da es mehrere Exemplare von Büchern geben kann, die verschiedene Besitzer und auch Zustände haben können. Eine weitere Entscheidung war, die Bewertung an die Ausleihe zu hängen. Durch Fremdschlüsselbeziehungen können nur Ausleiher einen Verleiher bewerten, von dem sie tatsächlich ein Buch ausgeliehen haben.

Der komplette Code sowie die SQL-Dokumentation sind auf GitHub zu finden:

https://github.com/FelixStellbrink/BuchtauschApp-DLBDSPBDM01

2. Funktionalität

Eine der zentralen Funktionalitäten ist die der Constraints. Das sind Regeln, die PostgreSQL bei jedem INSERT oder UPDATE automatisch prüft. Bei einem Verstoß erscheint eine Fehlermeldung und die Daten werden nicht gespeichert. Genutzt wurden hier Primärschlüssel, die jede Zeile eindeutig machen. In der Datenbank existieren zwei natürliche Schlüssel: die Postleitzahl von Ort und die ISBN von Buch. Die Postleitzahl wird mit einem CHAR angelegt, damit führende Nullen gespeichert werden. Außerdem gibt es zwei zusammengesetzte Primärschlüssel in den Tabellen buch_genre und buch_autor. Dies sorgt dafür, dass dieselbe Kombination nur einmal existiert. Der Rest der 13 Tabellen hat künstliche Schlüssel, die durch SERIAL automatisch in aufsteigender Reihenfolge vergeben werden. Ein weiterer Constraint ist der CHECK, der nur die eigene Zeile darauf prüft, ob bestimmte Werte enthalten sind. Beispielsweise muss eine E-Mail in der Tabelle nutzer ein „@“ enthalten. Auch wird geprüft, ob eine Bewertung zwischen 1 und 5 Sternen liegt. Ein Sonderfall ist die Prüfung von Startdatum ≤ Enddatum, die zwei Spalten derselben Zeile vergleicht. Fremdschlüssel sind Verweise auf andere Tabellen. Eine Besonderheit gibt es bei buch_genre und buch_autor: Wird ein Buch gelöscht, wird durch die Option ON DELETE CASCADE auch die Zuordnung gelöscht. Bei Exemplar hingegen gibt es kein CASCADE, daher wird das Löschen eines Buches verhindert, solange noch Exemplare daran hängen. UNIQUE prüft, dass ein Wert in der Spalte nur einmal vorkommen darf. Beispiele sind die E-Mail, die Sprache und dass pro Ausleihe nur eine

Bewertung existiert. UNIQUE gilt immer für alle Zeilen. Die Regel „nur eine laufende Ausleihe pro Exemplar“ lässt sich damit nicht ausdrücken.

Eine weitere Funktionalität sind die partiellen Indizes. Wo UNIQUE an seine Grenzen stößt, vergleicht ein Index in Kombination mit einer WHERE-Bedingung nur bestimmte Zeilen.

Der Index ausleihe_aktiv prüft in der Tabelle ausleihe, welche Ausleihen aktiv sind. Dies macht er durch die Bedingung WHERE enddatum IS NULL. Pro Exemplar darf höchstens eine laufende Ausleihe existieren. Ein Exemplar darf nacheinander beliebig oft verliehen werden.

In der Tabelle anfrage überprüft der Index anfrage_angenommen, ob höchstens eine angenommene Anfrage pro Exemplar existiert. Die Bedingung hierzu ist WHERE status = 'Angenommen'. Mehrere Nutzer dürfen dasselbe Exemplar anfragen, und der Verleiher entscheidet dann, welche Anfrage er annimmt. Ein normales UNIQUE würde hier nicht funktionieren, da jedes Exemplar nur ein einziges Mal ausleihbar bzw. anfragbar wäre. Eine Historie wäre dementsprechend unmöglich. Auch ein CHECK wäre nicht möglich, da dieser nur die eigene Zeile sehen kann und nicht mit anderen vergleichen.

Eine wichtige Funktionalität sind Trigger. Im Gegensatz zu einem Index kann ein Trigger Informationen aus anderen Tabellen lesen. Ein Trigger läuft automatisch vor oder nach einem INSERT oder einem UPDATE. BEFORE-Trigger prüfen und können ablehnen; AFTER-Trigger reagieren auf eine erfolgreiche Änderung.

Der erste ist trg_verfuegbarkeit. Er läuft in der Tabelle ausleihe und liest den Status in exemplar, bevor etwas hinzugefügt wird. Er lehnt eine Ausleihe ab, wenn ein Exemplar nicht verfügbar ist. Ein weiterer ist trg_bewertung. Er läuft in der Tabelle bewertung vor jedem INSERT und UPDATE und liest das Enddatum in ausleihe. Der Trigger lehnt eine Bewertung ab, wenn die Ausleihe noch läuft.

Um zu verhindern, dass ein Nutzer sein eigenes Exemplar anfragen kann, ist der Trigger trg_anfrage zuständig. Er liegt in der Tabelle anfrage und läuft vor jedem INSERT und UPDATE. Der Trigger vergleicht den anfragenden Nutzer mit dem Besitzer in exemplar.

Der letzte Trigger ist trg_status. Er ist der einzige Trigger mit AFTER INSERT und UPDATE. Er lehnt nichts ab, sondern ändert selbst die Daten. Wenn eine neue Ausleihe angelegt wird, wird das Exemplar auf Verliehen gesetzt. Wenn ein Enddatum gesetzt wird, wird das Exemplar zurück auf Verfügbar gesetzt. Archivierte Exemplare bleiben davon unberührt. Hier ist ein AFTER eingesetzt worden, da Trigger in derselben Tabelle alphabetisch ablaufen. Als BEFORE-Trigger käme trg_status vor trg_verfuegbarkeit. Er würde das Exemplar schon auf Verliehen setzen, und die Prüfung würde dann jede Ausleihe ablehnen. trg_verfuegbarkeit verlässt sich zusätzlich auf die Spalte exemplar.status. Wäre der Status falsch, würde er eine zweite Ausleihe zulassen. Der Index ausleihe_aktiv prüft dagegen die Ausleihen selbst. Dadurch entsteht eine doppelte Absicherung.

Die letzte Funktionalität sind Abfragen. Neben dem Schutz vor falschen Daten kann eine Datenbank Fragen beantworten. Die Abfragen verändern keine Daten, sondern zeigen, was in der Datenbank steht.

Die erste Abfrage (A1) zeigt die laufenden Ausleihen je Nutzer. Durch einen LEFT JOIN können auch Nutzer ohne laufende Ausleihen erscheinen. Die Bedingung enddatum IS NULL steht im ON, nicht im WHERE, sonst würden diese Nutzer wieder herausfallen. COUNT zählt ausleih_id anstatt aller Zeilen, da sonst jeder Nutzer mindestens eine Ausleihe bekommen würde.

Die Abfrage überfällige Ausleihen (A2) verbindet vier Tabellen miteinander. Die Fälligkeit wird mit dem Startdatum und der Ausleihdauer berechnet. Dieses Datum wird mit CURRENT_DATE verglichen, daher hängt das Ergebnis vom Ausführungstag ab.

Um eine Durchschnittsbewertung je Verleiher zu sehen (A3), werden auch vier Tabellen benötigt. Für den Durchschnitt wird AVG genutzt, und in Kombination mit COUNT kann auch die Anzahl der Bewertungen angezeigt werden. Die Tabellen ausleihe und exemplar haben beide eine nutzer_id, aber für verschiedene Personen, also Ausleiher und Besitzer. Bewertet wird der Besitzer, deshalb läuft der JOIN über exemplar.nutzer_id.

Die letzte Abfrage (A4) läuft über sechs Tabellen. Sie kann verfügbare Bücher nach Genre und Standort anzeigen. Ein Exemplar zeigt in zwei Richtungen: einmal zum Buch für Titel und Genre, einmal zum Besitzer für den Ort. STRING_AGG fasst mehrere Genres in einer Zeile zusammen. Sonst würde ein Buch mit mehreren Genres mehrfach erscheinen. Sortiert wird nach Postleitzahl. Entfernungen werden aber nicht berechnet, da dafür Geodaten nötig wären.

3. Metadaten

	tabelle text	eintraege bigint
1	anfrage	11
2	ausleihe	13
3	autor	12
4	bewertung	10
5	buch	12
6	buch_autor	13
7	buch_genre	18
8	exemplar	10
9	genre	13
10	nutzer	10
11	ort	10
12	sprache	12
13	uebergabeart	2

Abb. 1: Datenbestand. Quelle: Eigene Darstellung.

Die gesamte Größe der Datenbank beträgt 9342 kB. Ermittelt wurde dieser Wert mit der Funktion „pg_database_size('buchtausch')“. Dieser Wert umfasst die gesamte Datenbank einschließlich der Systemkataloge, die PostgreSQL in jeder Datenbank anlegt und die hier den Großteil ausmachen. Die erstellten Tabellen haben eine Größe von 440 kB. Ermittelt wurde dieser Umfang mit „pg_total_relation_size“. PostgreSQL speichert in Seiten zu je 8 kB und jede Tabelle und jeder Index belegt mindestens eine Seite. Die Datenbank besteht daher fast nur aus Struktur und kaum aus Datenvolumen.

In Abb. 1 sind alle 13 Tabellen aufgelistet. Sie haben insgesamt 146 Einträge. Jede Tabelle hat mindestens 10 Einträge. Die einzige Ausnahme bildet uebergabeart mit nur zwei Einträgen. Mehr Einträge wären hier nicht sinnvoll, da die Übergabe nur über Versand oder Abholung möglich ist.